RESULTADOS PRELIMINARES • PILOTO

Código de agente envelhece melhor?

Fluxo Autônomo × Humano no Loop em repositórios reais: complexidade, manutenibilidade, estrutura e retrabalho.

Tópicos em Informática e Sociedade · PUC · 15/10/2026 · Dataset AIDev

A literatura se contradiz

Sobrevive mais

Rahman & Shihab (2026): HR 0,84 para código de agente.

Sobrevive menos

Popescu et al. (2026): mais churn, menor durabilidade.

- Definições difusas de “código de IA”: autocompletar, chat e agente no mesmo balde.
- Comparações entre projetos diferentes, sem controle de tamanho nem de tarefa.
- Nossa aposta: comparar **fluxos de trabalho** dentro do **mesmo repositório**.

§1.2 e §1.6 da metodologia v7

O construto: fluxo, não autoria

	FA — Fluxo Autônomo	HL — Humano no Loop
Quem conduz	Agente, da tarefa à PR	Uma pessoa, do início ao fim
Uso de IA	Total	Permitido, sob condução humana
Revisão humana	Faz parte do fluxo	Faz parte do fluxo
O que descaracteriza	Humano commitar na PR	—

Unidade de análise: cada PR integrada à branch principal — o diff líquido que ela trouxe.

ÁRVORE GQM

Objetivo → 4 perguntas → 4 métricas primárias

G. Avaliar se o código do fluxo autônomo difere do código do fluxo com humano no loop, dentro do mesmo repositório.

Q1
Complexidade
cc_delta_sum

Q2
Manutenibilidade
mi_delta_mean

Q3
Estrutura e tamanho
lloc_delta

Q4
Correção e retrabalho
rework_rate_w

Toda métrica é reportada bruta e normalizada por tamanho · §6.4.3

A pipeline, em seis etapas

E1	Selecionar repositórios do AIDev onde os dois fluxos convivem
E2	Percorrer o histórico de 1º pai e classificar cada PR em FA ou HL
E3	Filtrar: só código de produção, 3 ≤ tamanho ≤ 500, sem reformatação
E4	Medir com Radon o código antes (B) e depois (A) de cada PR
E5	**git blame -w -M -C**: quem retrabalha as linhas, e quando (janela de 90 dias)
E6	Wilcoxon pareado por repositório + modelos mistos + Kaplan-Meier e Cox

Python 3.13 · Radon 6.0.1 · versões e parâmetros fixados em manifesto

O piloto: do dataset às unidades

6.673
repositórios no AIDev

24
na amostra Python

257
PRs FA elegíveis

294
unidades analisadas
18 FA · 276 HL

- P1 completa nos 24 repositórios; P2 e E6 em 3 do piloto, **~7 minutos** de execução.
- Dois repositórios entraram na análise: **crewAI** e **microsoft/lisa**.
- A metodologia rodou **inteira, ponta a ponta** — é esse resultado que apresentamos.

Corte do AIDev: 24/10/2025 · janela de retrabalho: 90 dias

Q1, Q2 e Q3: sem diferença detectável

QP	Métrica primária	FA	HL	Modelo (p)
Q1	cc_delta_sum	3,5	3,0	+2,04 · p = 0,49
Q2	mi_delta_mean	−0,44	−0,45	−0,18 · p = 0,86
Q3	lloc_delta	11,5	8,0	+0,43 · p = 0,96
Q3	size_churn	25	45	−48 linhas · p = 0,02

- Medianas por unidade; intervalos de confiança largos — **n = 18 FA**.
- O único efeito claro: **PRs de agente são menores** (metade do churn, 1 arquivo contra 2).
- Converge com *More Code, Less Reuse*: diferenças mínimas em complexidade.

LMM com intercepto aleatório por repositório · correção de Holm

Q4 — CORREÇÃO E RETRABALHO

O código de agente sobreviveu mais

61%

das linhas **FA** sem retrabalho em 90 dias

34%

das linhas **HL** sem retrabalho em 90 dias

0,56

HR de retrabalho (Cox)
IC 0,23 – 1,34

- Correções específicas (*fix*): HR 0,49 · IC 0,24 – 1,01 · **p = 0,053**.
- Mediana de sobrevivência do HL: **37 dias**. O FA não chegou à mediana na janela.
- Mesma direção de Rahman & Shihab — mas o IC cruza 1: **ainda não é conclusão**.

Kaplan-Meier e Cox com variância robusta por repositório · n = 294

O que o piloto quebrou

Perdemos 53% das PRs FA

O teste de consistência usa Jaccard; agentes tentam e desfazem dentro da própria PR. Correção: usar **contenção** de arquivos.

Rebase invisível

No **microsoft/lisa**, 69% dos eventos não têm marcação de PR. O HL perde PRs humanas reais.

A amostra não se mantém

De ~80 repositórios para **~35–50**, quase todos TypeScript. Python sozinho não fecha o N.

Oito recomendações registradas (R-P1 a R-P8). Nenhuma aplicada sem decisão do grupo.

Documento 04-analise-piloto.md · §4 e §5

Próximos passos

1. Corrigir a consistência (Jaccard → contenção) e reprocessar os 3 repositórios
2. Fixar as métricas primárias normalizadas por tamanho (pendência P3)
3. Implementar JavaScript/TypeScript com Lizard e rodar o piloto TS
4. Decidir a pendência P1: **Q4 vira a questão principal do artigo?**

Nosso enquadramento: reconciliar a contradição Rahman & Shihab × Popescu et al., com controle de tamanho e de tipo de tarefa.

Obrigado — perguntas?